

Programming Assignment 1: FOCC and BOCC

Aayush Kumar - CO21BTECH11002

Contents

1	Introduction	2
2	Common Test Framework	2
2.1	Initialization and Parameters	2
2.2	Transaction Execution	3
2.3	Timing and Metrics	3
2.4	Logging and Output	3
2.5	Modular Design	3
3	Basic Timestamp Ordering (BTO)	3
3.1	Data Model	4
3.2	Transaction Initialization	4
3.3	Read Operation	4
3.4	Write Operation	4
3.5	Commit Procedure	4
3.6	Logging	5
4	Multiversion Timestamp Ordering (MVTO)	5
4.1	Data Structure	5
4.2	Initialization	5
4.3	Reading	5
4.4	Writing	6
4.5	Commit and Abort	6
4.6	Logging	6
5	MVTO with Garbage Collection (MVTO-GC)	6
5.1	Garbage Collection Design	7
5.2	Implementation Details	7
5.3	K-Version MVTO (K-MVTO)	7
5.3.1	Version Management with Bounded Versions	7
5.3.2	Read and Write Conditions	8
5.3.3	Abort Handling and Version Eviction	8
6	Experiments	8
6.1	Experiment 1: Varying Number of Transactions	9
6.1.1	Observations	9
6.2	Experiment 2: Varying Database Size	10
6.2.1	Observations	11
6.3	Experiment 3: Varying Number of Threads	11

6.3.1	Observations	11
6.4	Experiment 4: Varying Bound on Number of Versions (Only for K-MVTO) . .	12
6.4.1	Observations	12
6.5	Experiment 5: Varying Probability of Read-Only Transactions	13
6.5.1	Observations	13

7	Conclusion	14
---	------------	----

1 Introduction

This report presents the implementation and comparative evaluation of four timestamp-based concurrency control protocols:

1. **Basic Timestamp Ordering (BTO):** A single-version protocol where each transaction is assigned a timestamp at the start, and all read and write operations are validated against this timestamp to enforce a global serializability order.
2. **Multiversion Timestamp Ordering (MVTO):** A multiversion concurrency control protocol that maintains multiple versions of each data item. Reads are served from appropriate versions based on transaction timestamps, significantly reducing abort rates by allowing more interleaved schedules.
3. **MVTO with Garbage Collection (MVTO-GC):** An extension of MVTO that incorporates garbage collection to remove obsolete versions and metadata associated with transactions that have completed. This helps manage memory usage effectively in long-running systems.
4. **K-Version MVTO (K-MVTO):** A bounded-version variant of MVTO where each data item retains at most K versions. Older versions are discarded in a controlled manner, balancing memory overhead with concurrency.

Each protocol was implemented in C++ using a common transactional interface. The underlying test driver executes concurrent transactional workloads and measures key performance metrics such as average commit time and abort rates.

2 Common Test Framework

All concurrency control protocols—Basic Timestamp Ordering (BTO), and the three Multiversion Timestamp Ordering (MVTO) variants—are evaluated using a common test framework. The only difference between each test case lies in the transaction management implementation, which is included via the respective header file (e.g., `bto.h`, `mvto.h`, etc.).

2.1 Initialization and Parameters

The program begins by reading benchmark parameters from an input file `inp-params.txt`. These parameters include:

- `n`: Number of threads.
- `m`: Size of the shared memory array.
- `totTrans`: Total number of transactions across all threads.

- `constVal`: Upper bound on the random value used during write operations.
- `lambda`: Mean of the exponential distribution used to simulate work delay.
- `numIters`: Number of operations per transaction.
- `readRatio`: Probability that a transaction is read-only.

2.2 Transaction Execution

Each thread executes a roughly equal share of the total transactions. A transaction may consist of several reads and optional writes, controlled by the `readRatio` parameter. For each operation:

1. A random index and value are selected.
2. A read is attempted from the shared memory. If the read is determined to be “late” by the concurrency control logic, the transaction is aborted.
3. If the transaction is not read-only, a write is also attempted.
4. Each operation is followed by a sleep period, drawn from an exponential distribution, to simulate delay and increase contention.

If a transaction is aborted, it is retried until it successfully commits. Upon successful commit or abort, a log entry is written detailing the event.

2.3 Timing and Metrics

Commit times are measured in nanoseconds using the `chrono` library, and each thread tracks its total commit time and the number of aborts. After all threads complete their transactions, the average commit time and number of aborts per transaction are reported.

2.4 Logging and Output

A log file records all read, write, and commit/abort events with timestamps, transaction IDs, and thread IDs. This aids in understanding protocol behavior under concurrency.

2.5 Modular Design

The test framework is kept modular by abstracting all protocol-specific logic behind a common interface (e.g., `begin_trans()`, `read()`, `write()`, `tryCommit()`). Thus, only the header file needs to be changed to evaluate different protocols.

3 Basic Timestamp Ordering (BTO)

Basic Timestamp Ordering (BTO) is a classical concurrency control protocol that ensures serializability by using transaction timestamps to order operations. In our implementation, each transaction is assigned a unique timestamp at the beginning. This timestamp is used to determine whether the transaction can safely read from or write to a data item without violating the serializability constraints.

3.1 Data Model

Each memory location is represented by an `Item` object containing:

- `val`: the actual value stored at the memory location.
- `max_w`: the maximum timestamp of any transaction that has written to this location.
- `read_ts`: a set of timestamps of transactions that have read this item. This is stored as a `set` to efficiently retrieve and update the maximum read timestamp, especially when transactions abort.
- `lock`: a binary semaphore to synchronize access to each data item.

3.2 Transaction Initialization

Each transaction is assigned a unique identifier using a global atomic counter. This ID also acts as the transaction's timestamp. A transaction maintains two sets:

- `readSet`: a map from indices to values representing the data items read by the transaction.
- `writeSet`: a map from indices to values representing the data items the transaction intends to write to.

3.3 Read Operation

A read operation performs the following steps:

1. If the transaction has already written to the data item, it reads from its local `writeSet`.
2. Otherwise, it checks whether its timestamp is greater than or equal to `max_w` for that item. If so, it reads the current value and inserts its timestamp into the `read_ts` set.
3. If the read is successful, the value is added to the transaction's `readSet`.

3.4 Write Operation

Writes are deferred until the transaction commits. The `write()` function simply stores the update in the transaction's `writeSet`.

3.5 Commit Procedure

The commit protocol proceeds as follows:

1. All locks for items in the transaction's `readSet` and `writeSet` are acquired to prevent interference.
2. If the transaction was already marked as aborted due to a failed read, it removes its timestamp from all `read_ts` entries and releases all locks.
3. Otherwise, the protocol checks whether the transaction is eligible to write to each item in its `writeSet` by verifying:

$$\text{timestamp} \geq \max(\text{max_w}, \text{max_read_ts})$$

4. If this condition fails for any item, the transaction is aborted and its timestamp is removed from the relevant `read_ts` sets.
5. If eligible, the transaction applies all its writes to the database, updates `max_w`, and logs the write.
6. To limit the size of the `read_ts` sets, timestamps strictly less than the current transaction's ID are pruned.
7. All acquired locks are released, and the transaction is deleted.

3.6 Logging

The system also logs committed writes with timestamps for debugging and analysis. This is synchronized using a semaphore `logLock` to prevent interleaved output.

4 Multiversion Timestamp Ordering (MVTO)

Multiversion Timestamp Ordering (MVTO) is a concurrency control protocol that allows multiple versions of a data item to exist in the database. It ensures serializability by associating a unique timestamp with each transaction and controlling the read and write operations based on these timestamps. MVTO avoids conflicts by ensuring that a transaction reads the most recent version of a data item that precedes its own timestamp and checks for serialization-violating writes before committing.

4.1 Data Structure

Each data item maintains multiple versions. A version is represented as a pair: a value written by a transaction and a set of transaction IDs (timestamps) that have read this version. The versions are stored in a sorted map from timestamps to version records:

- `versions: map<timestamp, (value, read_list)>`

A global vector `shared` holds all the data items.

4.2 Initialization

At the start, the database is initialized with m items. Each item has a default version written by a pseudo-transaction t_0 with timestamp 0. This ensures that a read is always possible, as required by the MVTO protocol.

```
shared[i].versions[0] = {getRand(0, INF), {}};
```

4.3 Reading

To read an item:

- If the transaction has previously written to the item, it reads from its local write set.
- Otherwise, the most recent version with timestamp $\leq$ the transaction's timestamp is fetched from the `versions` map.

- The transaction's timestamp is added to the read list of this version.

The read operation always succeeds since the version created by transaction t_0 guarantees at least one readable version.

4.4 Writing

A transaction locally buffers its writes in a `writeSet` without affecting the database immediately. During commit, it checks if it can safely write the buffered values based on MVTO rules:

1. A valid version with timestamp $\leq$ transaction timestamp must exist (this is always true because of t_0).
2. The transaction's timestamp must not fall between the write timestamp ts_i of an existing version and the read timestamp ts_j of any transaction that read that version:

$$\text{Invalid if } ts_i < ts < ts_j$$

To enforce this, for each item in the write set:

```
iter = versions.lower_bound(timestamp);
iter--; // version with largest ts < timestamp
ll max_ts = max(iter->second.second);
```

If the above condition fails for any item, the transaction aborts.

4.5 Commit and Abort

Before committing, the transaction locks all items it has read or written. If all write checks pass, the transaction:

- Appends new versions to the corresponding data items.
- Logs its writes with precise timestamps.

If the commit is not possible, the transaction:

- Removes itself from the read lists of versions it accessed.
- Releases all locks and deletes itself.

4.6 Logging

The system also logs committed writes with timestamps for debugging and analysis. This is synchronized using a semaphore `logLock` to prevent interleaved output.

5 MVTO with Garbage Collection (MVTO-GC)

MVTO-GC builds on the standard MVTO algorithm by introducing a simple but effective garbage collection mechanism that reclaims obsolete versions of data items. This section describes only the additions and differences relative to the basic MVTO model.

5.1 Garbage Collection Design

The central idea is that once all transactions with timestamps earlier than a given version are no longer active, that version (and any older versions) can be safely discarded—except the latest version that might still be relevant for validating newer transactions.

Let T_{min} be the minimum timestamp among all currently active transactions. For each data item, MVTO-GC identifies the latest version v with timestamp $t < T_{min}$ and retains it. All versions with timestamps smaller than t are no longer needed, as:

- They will not be read by any future transaction (since all future transactions have timestamps $\geq T_{min}$).
- They are not needed to validate writes by any future transaction (which only check for conflicts against the latest version before T_{min}).

5.2 Implementation Details

Tracking Active Transactions A global set `activeTrans` maintains timestamps of all currently active transactions. Each transaction adds its timestamp on creation and removes it upon commit or abort. This set is protected by a semaphore `activeLock`.

Version Cleanup Procedure Garbage collection is invoked at the end of a successful commit, during the critical section where the transaction holds locks on its read and write sets. The `collectGarbage()` function performs the following steps for each data item accessed by the committing transaction:

1. Determine T_{min} , the minimum timestamp in `activeTrans`.
2. Identify the latest version v such that `timestamp(v) < Tmin`.
3. Delete all versions older than v .

This ensures correctness while keeping version history bounded. Performing garbage collection inside the critical section (after commit and before releasing locks) guarantees consistency and thread-safety.

5.3 K-Version MVTO (K-MVTO)

The K-MVTO algorithm is a variant of the basic MVTO protocol that restricts the number of versions maintained per data item to a fixed constant K . This section describes only the additions and differences relative to the basic MVTO model.

5.3.1 Version Management with Bounded Versions

In K-MVTO, each data item maintains a map from timestamps (transaction IDs) to versions. Each version is a pair consisting of the value written and the set of transaction IDs that have read that version:

`map<timestamp, (value, read_list)>`

When a transaction writes to an item, it creates a new version. If the number of versions exceeds K , the oldest version is evicted to make room for the new one. This removal policy ensures that the version list remains bounded:

- If `versions.size() < K`, insert the new version normally.
- Otherwise, erase `versions.begin()` (the oldest version) before inserting the new version.

5.3.2 Read and Write Conditions

The conditions for reading and writing are slightly adapted to handle possible version evictions:

Read Condition A transaction T can read from a data item only if a version exists with timestamp less than or equal to T 's timestamp. If the earliest version's timestamp exceeds T 's timestamp, then T must abort. This implies that if older versions are removed and T 's read timestamp is too early, it cannot proceed:

$$\text{canRead}(T) : T.\text{id} \geq \min(\text{versions.keys}())$$

Write Condition A transaction T can write to a data item only if two conditions are satisfied:

1. T 's timestamp is no smaller than the smallest timestamp in the version map (same as read condition).
2. Let V be the latest version with timestamp less than T 's. Let $\max R$ be the largest timestamp of any transaction in V 's read list. Then:

$$\neg (V.\text{timestamp} < T.\text{id} < \max R)$$

This prevents scenarios where a newer transaction T might violate serialization by interleaving between a writer and its readers.

5.3.3 Abort Handling and Version Eviction

If a transaction is aborted (either explicitly or due to failed write conditions), it must:

- Remove itself from the `read_list` of all versions it read.
- Note that the version it read might no longer exist (due to version eviction). In this case, attempting to remove itself from the `read_list` is safe, as the `erase` operation on sets is idempotent.

This logic ensures correctness even in the presence of evicted versions.

6 Experiments

The experiments were performed on **11th Gen Intel(R) Core(TM) i5-11300H @ 3.10GHz** with **8GB RAM** and **8 cores**. The following parameters remain constant in all experiments:

- **Constant Value** (*constVal*): 1000
- **Lambda** (λ): 20
- **Number of iterations per transaction** (*numIters*): 20

Transactions	BTO	MVTO	MVTO-GC	K-MVTO
1000	708796999.3	682770560.5	683514762.8	707992718.1
2000	712560371	693498238.5	686301638.5	697612413.4
3000	711742209.8	693600195.1	687075706	704538128.1
4000	718987964.5	710077234.7	711163748.3	713954170.6
5000	715186780.2	701671301.4	705388740	710512102.3

Table 1: Execution Time for Varying Number of Transactions

Transactions	BTO	MVTO	MVTO-GC	K-MVTO
1000	0.758	0.685	0.691	0.746
2000	0.765	0.686	0.675	0.73
3000	0.763	0.692	0.698	0.743
4000	0.741	0.697	0.702	0.716
5000	0.772	0.685	0.698	0.711

Table 2: Number of aborts for Varying Number of Transactions

For K-MVTO, the bound on the number of versions is set to 5, except in the 4th experiment where it is varied from 5 to 25. All the values of time are in nanoseconds. The average time taken by a transaction to commit and average number of aborts per transaction are measured for each experiment. The results are shown in the tables and figures below.

6.1 Experiment 1: Varying Number of Transactions

We fix the number of threads (n) at 8, database size (m) at 1000, read only probability ($readRatio$) at 0.7, and vary the total number of transactions ($numTrans$) from 1000 to 5000 and measure the average time taken by a transaction to commit and average number of aborts per transaction. The results are shown in Table 1, 2 and Figure 1.

6.1.1 Observations

- The average time taken and the average abort count per transaction remains almost constant with the number of transactions for all the algorithms. This is since the performance is determined mainly by the number of concurrent transactions and not the total number of transactions. As the number of transactions increases, total time taken increases linearly but the average time per transaction remains constant.

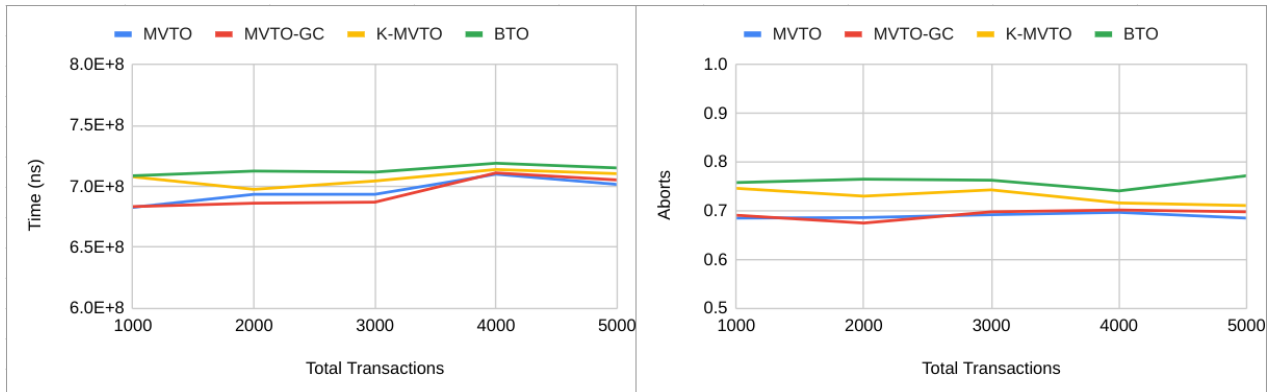


Figure 1: Execution Time and Aborts for Varying Number of Transactions

Transactions	BTO	MVTO	MVTO-GC	K-MVTO
1000	737861124.6	688278532.2	689760822.8	712874571.8
2000	556815464.6	526350833.2	523915258.5	535423557.4
3000	484000056.2	479425196.9	478919242.3	477649710.4
4000	462325949.9	460649048.5	455792568.5	454283269.6
5000	447848113.9	444047512.4	444840960.1	444562896.6

Table 3: Execution Time for Varying Number of Transactions

Transactions	BTO	MVTO	MVTO-GC	K-MVTO
1000	0.749	0.714	0.709	0.729
2000	0.311	0.294	0.301	0.308
3000	0.203	0.192	0.189	0.198
4000	0.147	0.125	0.124	0.133
5000	0.112	0.099	0.101	0.107

Table 4: Number of aborts for Varying Number of Transactions

- MVTO variants (MVTO, MVTO-GC, K-MVTO) have a lower average time per transaction and abort rate compared to BTO. K-MVTO has a slightly higher average time per transaction and abort rate compared to MVTO and MVTO-GC. This is because K-MVTO has a bound on the number of versions it can maintain, which can lead to more aborts if a transaction tries to read an older version that has been evicted.
- Time per transaction for MVTO-GC and K-MVTO are almost the same. This is because the garbage collection overhead is causing the time taken to increase slightly.

6.2 Experiment 2: Varying Database Size

We fix the number of threads (n) at 8, total number of transactions ($numTrans$) at 1000, read only probability ($readRatio$) at 0.7, and vary the database size (m) from 1000 to 5000. We measure the average time taken by a transaction to commit and average number of aborts per transaction. The results are shown in Table 3, 4 and Figure 2.

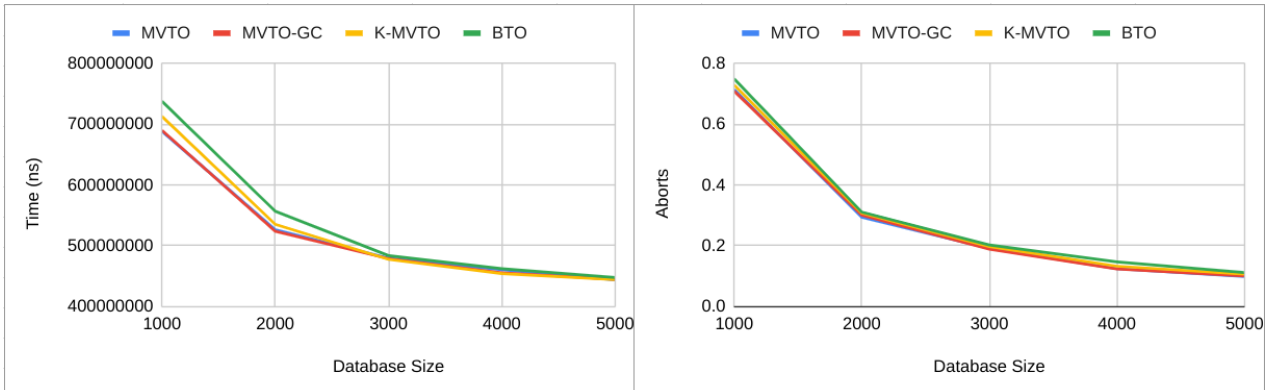


Figure 2: Execution Time and Aborts for Varying Database Size

Transactions	BTO	MVTO	MVTO-GC	K-MVTO
2	452394687.5	436674336	442438341.6	447908523.7
4	535201669.4	508573396.9	511111035.9	515687662
8	727247514.7	699101855.4	713233656	719499714
16	1447358009	1325380393	1399222735	1403824135
32	6867628726	6008699129	6393705327	6552207843

Table 5: Execution Time for Varying Number of Transactions

Transactions	BTO	MVTO	MVTO-GC	K-MVTO
2	0.075	0.063	0.067	0.071
4	0.254	0.232	0.227	0.233
8	0.775	0.734	0.732	0.765
16	2.736	2.284	2.312	2.571
32	15.316	13.987	14.121	15.063

Table 6: Number of aborts for Varying Number of Transactions

6.2.1 Observations

- The average time taken and the average abort count per transaction decreases with the database size for all the algorithms. This is because a larger database size reduces the probability of conflicts between transactions.
- MVTO variants (MVTO, MVTO-GC, K-MVTO) have a lower average time per transaction and abort rate compared to BTO. K-MVTO has a slightly higher average time per transaction and abort rate compared to MVTO and MVTO-GC. This is because K-MVTO has a bound on the number of versions it can maintain, which can lead to more aborts if a transaction tries to read an older version that has been evicted.
- Time per transaction for MVTO-GC and K-MVTO are almost the same. This is because the garbage collection overhead is causing the time taken to increase slightly.

6.3 Experiment 3: Varying Number of Threads

We fix the database size (m) at 1000, total number of transactions ($numTrans$) at 1000, read only probability ($readRatio$) at 0.7, and vary the number of threads (n) from 2 to 32. We measure the average time taken by a transaction to commit and average number of aborts per transaction. The results are shown in Table 5,6 and Figure 3.

6.3.1 Observations

- The average time taken and the average abort count per transaction increases with the number of threads for both BOCC and FOCC. This is because more threads lead to increased probability of conflicts between transactions.
- For very small number of threads, the performance of all the algorithms is almost the same. This is because the number of transactions is small and hence the probability of conflicts is low and very less transactions get aborted. However, as the number of threads increases, the performance of BTO degrades significantly compared to MVTO variants. This is because BTO has a single version for each data item and hence it has

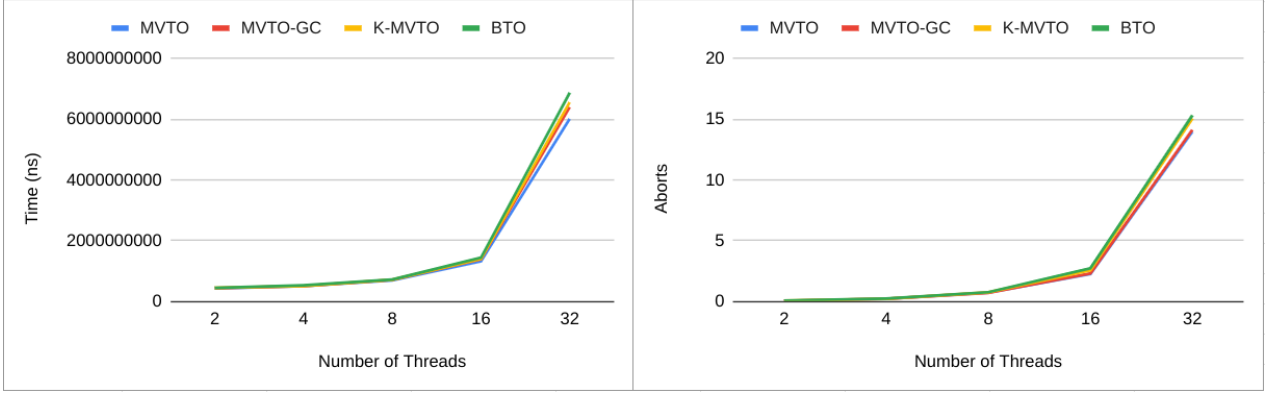


Figure 3: Execution Time and Aborts for Varying Number of Threads

Transactions	K-MVTO
5	700229504.8
10	695125979.8
15	681878042.1
20	683990655.4
25	684915017.9

Table 7: Execution Time for Varying Number of Transactions

to abort more transactions compared to MVTO variants which have multiple versions for each data item.

6.4 Experiment 4: Varying Bound on Number of Versions (Only for K-MVTO)

We fix the number of threads (n) at 8, database size (m) at 1000, total number of transactions ($numTrans$) at 1000, read only probability ($readRatio$) at 0.7, and vary the bound on the number of versions (K) from 5 to 25. We measure the average time taken by a transaction to commit and average number of aborts per transaction. The results are shown in Table 7,8 and Figure 4.

6.4.1 Observations

- As the bound on the number of versions increases, the average time taken and the average abort count per transaction decreases. This is because a larger bound allows more versions to be retained, reducing the probability of read operations failing due to version eviction.

Transactions	K-MVTO
5	0.739
10	0.718
15	0.681
20	0.684
25	0.686

Table 8: Number of aborts for Varying Number of Transactions

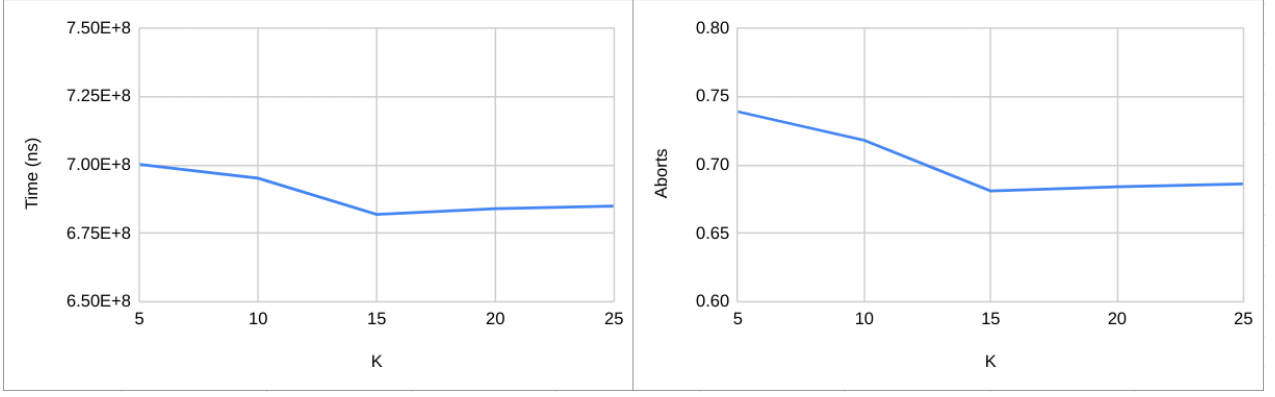


Figure 4: Execution Time and Aborts for Varying Number of Threads

Transactions	BTO	MVTO	MVTO-GC	K-MVTO
0.5	869274659.2	847157317.4	850268233.8	858788819.7
0.6	825669942.6	791611931.1	806818391.3	811973749.4
0.7	699707669.3	668647506.7	689863308.4	692984619.2
0.8	627013925	595779215.4	612068103	602098606.2
0.9	535377077.8	504512839	516237572.8	520996752.8

Table 9: Execution Time for Varying Number of Transactions

- For large values of K , the performance of K-MVTO approaches that of MVTO, as the eviction policy becomes less aggressive. However, the performance gain diminishes after a certain point, as seen in the results for $K = 20$ and $K = 25$. This is because the items are accessed randomly, and hence, on average, the number of versions created becomes smaller than K .

6.5 Experiment 5: Varying Probability of Read-Only Transactions

We fix the number of threads (n) at 8, database size (m) at 1000, total number of transactions ($numTrans$) at 1000, and vary the read only probability ($readRatio$) from 0.5 to 0.9. We measure the average time taken by a transaction to commit and average number of aborts per transaction. The results are shown in Table 9,10 and Figure 5.

6.5.1 Observations

- The average time taken and the average abort count per transaction decreases with the read-only probability for all the algorithms. This is because a higher read-only probability reduces the number of write operations, leading to fewer conflicts and aborts.

Transactions	BTO	MVTO	MVTO-GC	K-MVTO
0.5	1.255	1.117	1.107	1.168
0.6	0.992	0.929	0.922	0.946
0.7	0.734	0.663	0.673	0.719
0.8	0.499	0.462	0.467	0.481
0.9	0.297	0.256	0.252	0.281

Table 10: Number of aborts for Varying Number of Transactions

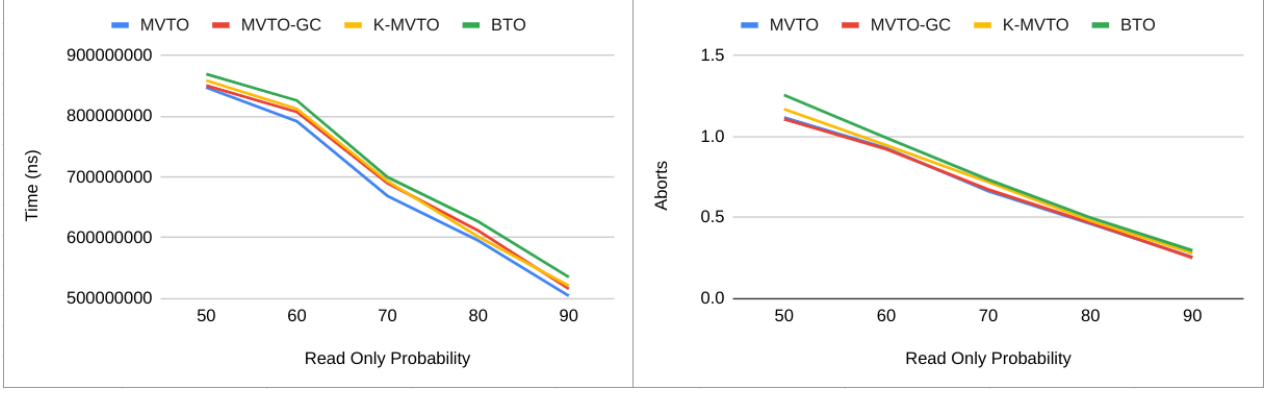


Figure 5: Execution Time and Aborts for Varying Number of Threads

- MVTO variants (MVTO, MVTO-GC, K-MVTO) have a lower average time per transaction and abort rate compared to BTO. K-MVTO has a slightly higher average time per transaction and abort rate compared to MVTO and MVTO-GC. This is because K-MVTO has a bound on the number of versions it can maintain, which can lead to more aborts if a transaction tries to read an older version that has been evicted.

7 Conclusion

Overall, the results show that MVTO-based algorithms consistently perform better than BTO in terms of both average time per transaction and abort rates. This is because MVTO variants allow multiple versions of data, which helps reduce conflicts between transactions.

More specifically:

- When the number of transactions increases, the average time and aborts per transaction remain nearly the same across all algorithms. This is expected, since performance depends more on how many transactions are active at the same time, rather than the total number of transactions.
- As the database size increases, both the execution time and abort rates decrease. This is because a larger database reduces the chance of conflicts.
- With more threads, all algorithms experience more conflicts and therefore higher abort rates and longer execution times. However, BTO suffers the most, as it only keeps a single version of each data item.
- In the case of K-MVTO, increasing the version bound (K) helps reduce aborts and improves performance—up to a point. Beyond a certain K , the benefit flattens out because most transactions don't need that many versions.
- When the read-only probability increases, performance improves significantly for all algorithms. Read-only transactions don't cause conflicts, so fewer transactions get aborted, and the system runs more efficiently.

In summary, MVTO variants offer a clear advantage over BTO, especially in scenarios with high concurrency or many read-only transactions. K-MVTO provides a good balance between performance and memory use, making it a practical choice when unlimited versioning is not feasible.